SRF Technical Reference: Implementation and Architectural Evolution

This document provides a comprehensive technical reference for the Structured Recomputation Framework (SRF), detailing the algorithmic transformations from storage-dominated dynamic programming to self-stabilizing, locality-aware, and rigorously validated deterministic recomputation.

SRF is an algorithmic memory restructuring framework, not a new algorithmic family. All transformations preserve mathematical correctness and biological validity.

1. Scientific Claims & Non-Claims

1.1 Structural Guarantees

- **Deterministic Recomputation:** SRF guarantees that every recomputed value is bit-identical to the value stored in a traditional full-matrix implementation.
- **Bounded Algorithmic Memory:** The algorithmic state is strictly bounded to $O(N)$ or $O(Width)$ depending on the algorithm, independent of total problem depth or area.
- **Platform Independence of Metrics:** Core intensity ratios (R_{mem}, R_{rec}) are proven to be platform-invariant algorithmic properties.

1.2 Non-Guarantees (Strict Prohibitions)

- **No Universal Speedup:** SRF does not guarantee that recomputation will be faster than baseline execution. Performance is a function of the memory-compute reciprocity.
- **No Big-O Improvement:** SRF does not reduce the asymptotic time complexity ($O(N^2)$ remains $O(N^2)$).
- **Hardware Sensitivity:** Wall-clock results are highly sensitive to specific L1/L2 cache geometries and memory bandwidth.

---

2. Phase 1: Baseline Architecture

Phase 1 established reference implementations to define ground truth behaviour, ensuring correctness and reproducibility across platforms.

2.1 Implementation Mechanics

Needleman–Wunsch (Global Alignment) Data Structure

```
std::vector<std::vector<int>> dp(N+1, std::vector<int>(M+1));
```

Recurrence Relation For sequences A and B:

$$dp[i][j] = \max \begin{cases} dp[i-1][j-1] + \text{match_or_mismatch}(A[i], B[j]) \\ dp[i-1][j] + \text{gap_penalty} \\ dp[i][j-1] + \text{gap_penalty} \end{cases}$$

2.2 Baseline Performance Metrics

Phase 1 established the performance floor across platforms: macOS (~1.1 MB floor), Linux (~3.4 MB), and Windows (~4.4 MB).

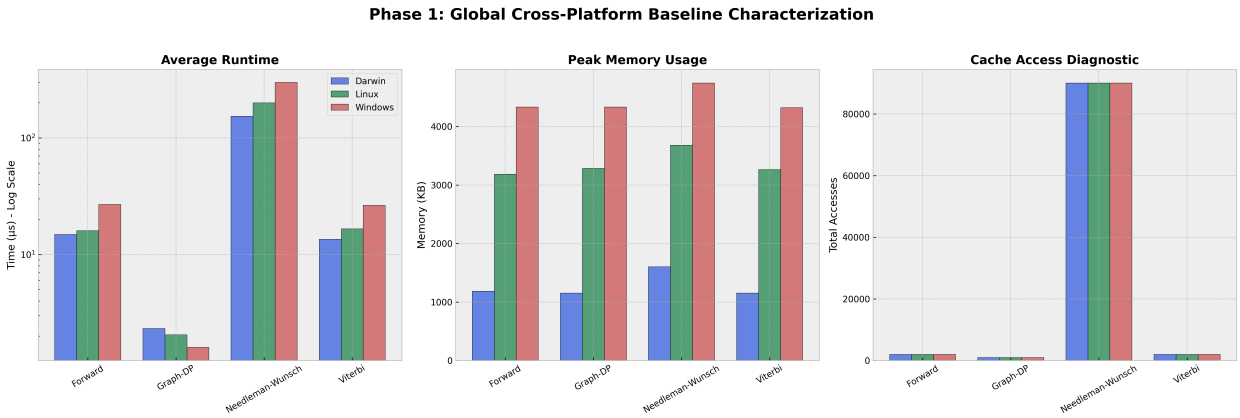

Figure 1: Phase 1 Global Master Profile

3. Phase 2: Functional SRF (Memory Restructuring)

Phase 2 replaced stored intermediate state with deterministic recomputation.

3.1 The O(N) Transition

- **Alternating Buffers:** Replaced $O(N \cdot M)$ matrices with two alternating linear buffers.
- **Memory Lifecycle:** Only row $(i-1)$ is retained during row i evaluation.
- **Checkpointing Strategy:** Stored boundary states at intervals (B or K) to enable recovery of transient interior states.

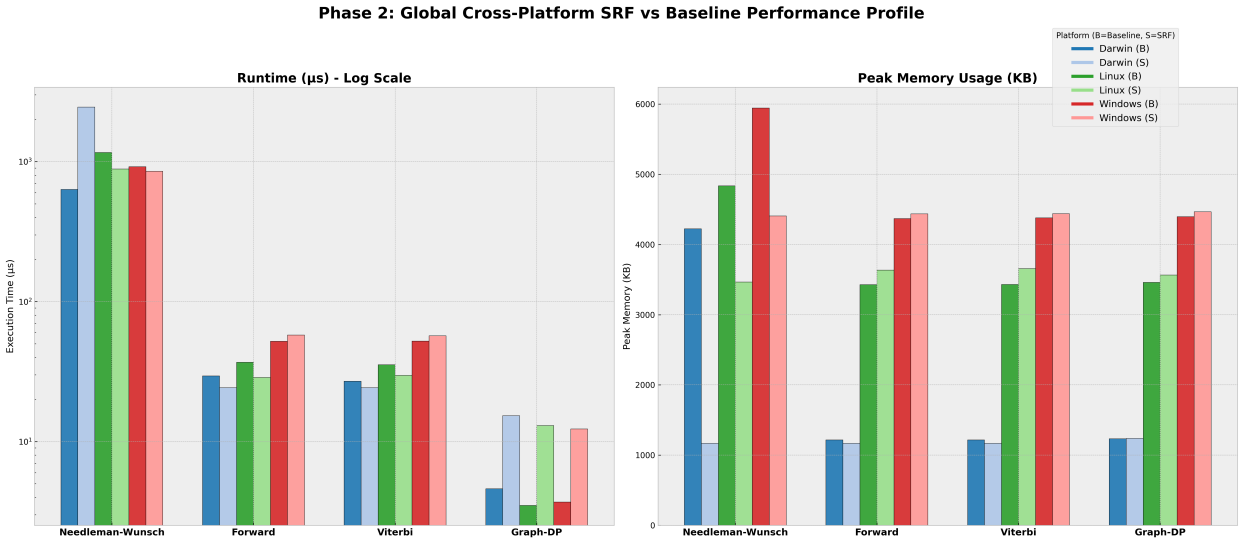

Figure 2: Phase 2 Global Master Profile

4. Phase 3: Locality Optimization

Phase 3 improved speed within the fixed $O(N)$ footprint by aligning recomputation with the CPU cache.

4.1 Cache-Aware Tiling

- **Strategy:** Align recomputation blocks with physical cache hierarchy.
- **Model:**

$$\text{TileSize}^2 \times 4 \text{ bytes} \leq \text{CacheBudget}$$

- **Finding:** Reducing DRAM traffic can offset recomputation costs, occasionally leading to baseline-parity performance on high-latency systems.

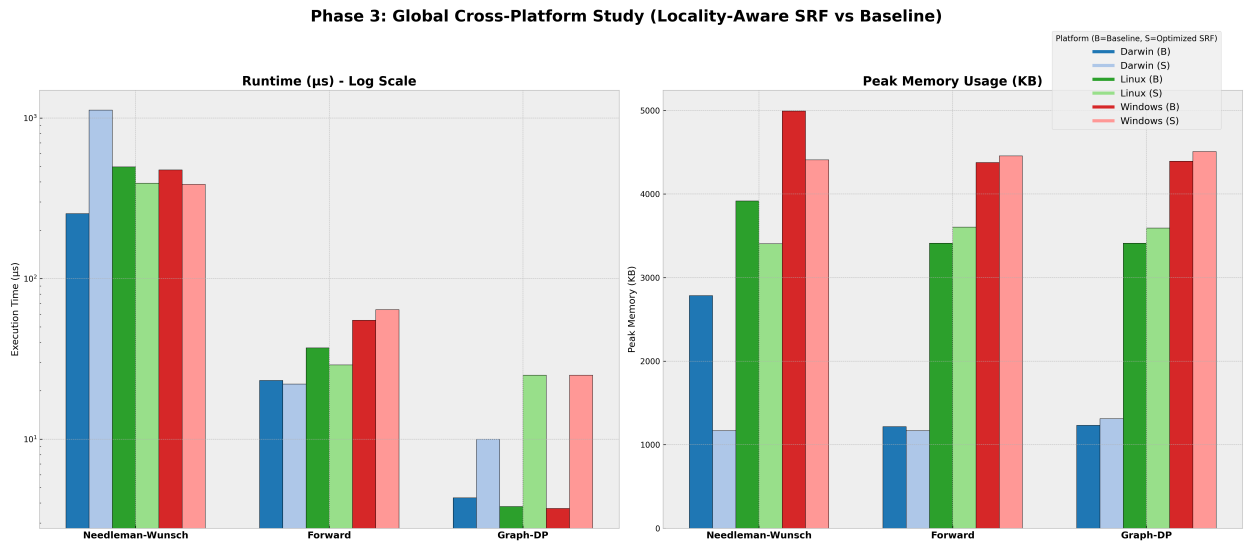

Figure 3: Phase 3 Global Master Profile

5. Phase 4 & 5: Backends & Granularity

Phases 4 and 5 introduced hardware abstraction and management amortization.

5.1 Backend Abstraction (`IBackend`)

SRF logic is decoupled from execution via a primitive API, ensuring portability across CPU and simulated GPU environments without semantic drift.

5.2 Granularity Amortization Law

Management overhead follows a strict **Inverse-Linear Scaling Law**:

$$\text{Management_Events} \propto \frac{1}{G}$$

Coarsening the recomputation unit ($G > 1$) reduces bookkeeping frequency by up to 100x.

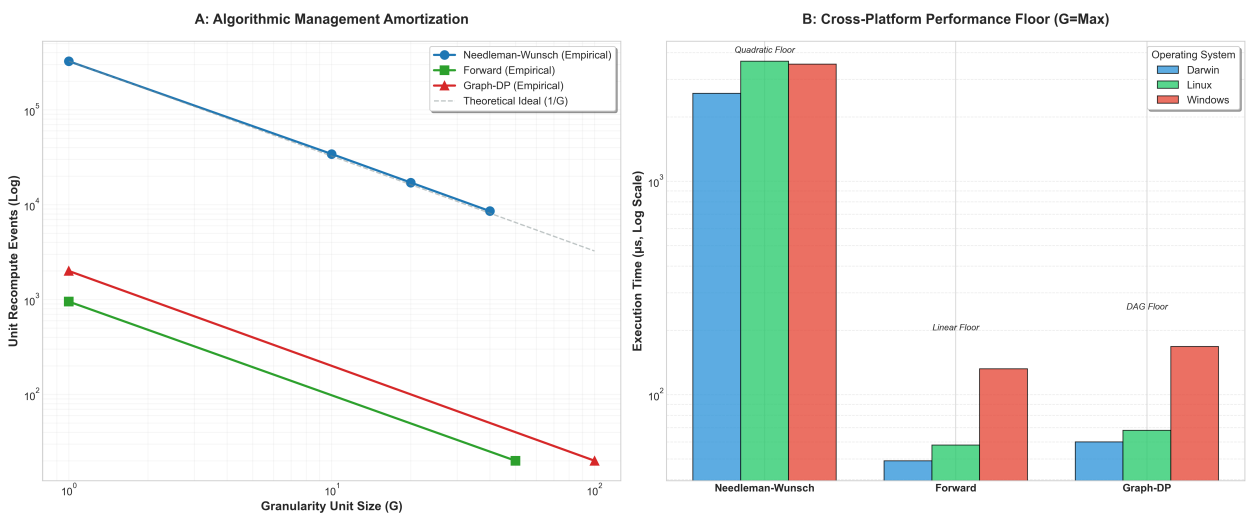

Figure 4: Phase 5 Global Master Profile

6. Regime Taxonomy & Analytical Modeling

6.1 Metric Interpretation Guide

- R_{mem} **(Memory Intensity):** Ratio of memory interactions to arithmetic compute. Values > 1.0 indicate bandwidth-bound behavior.
- R_{rec} **(Recomputation Intensity):** Ratio of redundant to base compute. High values signal recomputation dominance.
- **Locality Index:** Measure of unit-level reuse (Reuse/G).

6.2 Execution Regimes

1. **Memory-Intensive:** High R_{mem}, limited by buffer synchronization (e.g., NW at extreme scale).
2. **Balanced:** Linear scaling where compute and memory costs are amortized (e.g., Forward algorithm).
3. **Recomputation-Dominated:** High R_{rec}, limited by recursive state recovery latency (e.g., dense DAGs).

6.3 Model Validity Regions

Models are valid in the **deterministicRAM model**. Validity degrades in environments with non-deterministic hardware prefetchers or speculative execution that bypasses the structured recomputation schedule.

7. Robustness & Failure Mode Characterization

Phase 9 stress-validation identified specific failure regimes:

7.1 Degradation Regimes

- **Overhead Dominance:** In tiny workloads ($N < 100$), management logic (unit-ID lookups) dominates runtime.
- **Locality Collapse:** Occurs in dense topologies where dependency spans exceed cache capacity, forcing DRAM re-traversal for every recompute.

7.2 Determinism & Reproducibility

Algorithmic paths are 100% stable. Reproducibility is guaranteed by: 1. **Fixed-Seed Sequencing:** Identical input generation. 2. **Atomic Counter Tracks:** Bit-exact logging of recompute events. 3. **State Isolation:** Decoupling of algorithmic state from OS-level RSS.

---

9. CI / Workflow Philosophy

The SRF validation pipeline is designed for **continuous correctness and stability verification** rather than exhaustive performance benchmarking.

- **Scale Restrictions:** CI runners execute only **XS and S scales** to maintain bounded runtime and prevent resource exhaustion.
- **Fallback Strategy:** Extreme-scale tests are attempted with an automatic **Moderate Fallback** to ensure stable green builds while still probing hardware limits when possible.
- **Deterministic Verification:** Every push is validated for **bit-exact output equivalence** across macOS, Linux, and Windows, ensuring that architectural evolutions do not compromise mathematical integrity.

Conclusion: The Feasibility Envelope

SRF proves that **Memory Feasibility** is often distinct from **Runtime Feasibility**. While recomputation always introduces an arithmetic tax, it enables the execution of bioinformatics workloads that are otherwise hardware-impossible. The framework establishes a rigorous, self-stabilizing model for memory–time–locality interactions.